SkillForge Java Mastery

120 Most Common & Important Java Interview Questions with Answers

Set 1: Beginner + Intermediate — 60 Questions

This set is intentionally beyond basic syntax and includes OOP, collections, exceptions, streams, concurrency fundamentals, immutability and common interview concepts.

1. What is Java?

Java is a high-level, object-oriented programming language designed to be portable. Java source code is compiled into bytecode that runs on a Java Virtual Machine (JVM).

2. What are the main features of Java?

Key features include object-oriented programming, platform independence, automatic memory management, strong typing, exception handling, multithreading, security and a rich standard library.

3. Explain JDK, JRE and JVM.

JVM executes bytecode. **JRE** provides the runtime environment and libraries needed to run Java applications. **JDK** provides development tools such as the compiler plus runtime components.

4. Why is Java platform independent?

Java is compiled into platform-independent bytecode. A platform-specific JVM executes that bytecode, allowing the same compiled program to run across supported operating systems.

5. What is bytecode?

Bytecode is the intermediate instruction format produced by the Java compiler and stored in .class files. The JVM loads and executes it.

6. What is the difference between stack and heap memory?

The heap stores objects and is managed by garbage collection. Each thread has its own stack containing method frames, local variables and operand stacks.

7. What is a class and what is an object?

A class is a blueprint defining state and behaviour. An object is an instance of that class with its own state.

8. Explain the four pillars of OOP.

The four pillars are Encapsulation, Inheritance, Polymorphism and Abstraction. They help structure software around objects, contracts and reusable behaviour.

9. What is encapsulation?

Encapsulation combines data and behaviour inside a class while controlling access to internal state, commonly using private fields and public methods.

10. What is inheritance?

Inheritance allows a class to extend another class and reuse accessible state and behaviour. It should be used when a genuine 'is-a' relationship exists.

11. What is polymorphism?

Polymorphism allows the same method call or interface reference to work with different implementations. Overloading is compile-time polymorphism; overriding provides runtime polymorphism.

12. What is abstraction?

Abstraction exposes essential behaviour while hiding implementation details. Java supports it through interfaces and abstract classes.

13. Overloading vs overriding?

Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding provides a subclass implementation of an inherited instance method and is resolved dynamically.

14. What is a constructor?

A constructor initializes an object during creation. It has the same name as the class and no return type.

15. Can constructors be inherited or overridden?

No. Constructors are neither inherited nor overridden. A subclass constructor can invoke a superclass constructor using super().

16. What is the difference between == and equals()?

For objects, == compares references while equals() compares logical equality when appropriately overridden. For primitives, == compares values.

17. Why should equals() and hashCode() be overridden together?

If two objects are equal according to equals(), they must have the same hashCode(). This contract is essential for HashMap, HashSet and other hash-based collections.

18. Why is String immutable?

String immutability improves security, thread safety, caching and hash-code reliability. Once created, a String object's contents cannot be changed.

19. String vs StringBuilder vs StringBuffer?

String is immutable. StringBuilder is mutable and generally preferred for string construction in a single thread. StringBuffer is synchronized and is a legacy option when synchronized string mutation is specifically needed.

20. What is the String pool?

The String pool allows equal string literals to share objects, reducing duplicate memory usage. String literals are commonly interned by the JVM.

21. What are Java primitive types?

Java has eight primitive types: byte, short, int, long, float, double, char and boolean.

22. What is autoboxing and unboxing?

Autoboxing converts a primitive to its wrapper type, such as int to Integer. Unboxing converts a wrapper back to its primitive value.

23. Interface vs abstract class?

An interface primarily defines a contract and supports multiple inheritance of type. An abstract class can provide shared state and implementation. A class can extend one class but implement multiple interfaces.

24. What is final?

final can prevent variable reassignment, method overriding or class inheritance, depending on where it is applied.

25. What is static?

A static member belongs to the class rather than an individual object. Static fields are shared and static methods can be invoked without creating an instance.

26. What are access modifiers?

Java provides private, package-private, protected and public access levels. They control where classes and members can be accessed.

27. What is an exception?

An exception is an object representing an abnormal condition that can disrupt normal program flow and be handled by application code.

28. Checked vs unchecked exceptions?

Checked exceptions must be handled or declared by the compiler. Unchecked exceptions are RuntimeException subclasses and do not have that compile-time requirement.

29. throw vs throws?

throw explicitly throws an exception object. throws declares exceptions that a method may propagate to its caller.

30. What is try-with-resources?

Try-with-resources automatically closes AutoCloseable resources such as files and streams, making resource management safer and less error-prone.

31. What is the Java Collections Framework?

It is a set of interfaces and implementations for grouping and manipulating data, including List, Set, Queue and Map.

32. List vs Set vs Map?

List is ordered and usually allows duplicates. Set stores unique elements. Map stores key-value associations and is separate from the Collection hierarchy.

33. ArrayList vs LinkedList?

ArrayList provides fast indexed access and is usually the default List choice. LinkedList has linked nodes and can be useful for specific insertion/removal patterns, but indexed access is slow.

34. HashMap vs Hashtable?

HashMap is unsynchronized and permits null keys and values. Hashtable is a legacy synchronized class and rejects null keys and values. Modern code usually prefers HashMap or concurrent collections.

35. How does HashMap work at a high level?

HashMap uses a key's hash information to select a bucket and equals() to locate a matching key. Modern implementations can treeify heavily collided buckets.

36. What is HashSet?

HashSet stores unique elements using hashCode() and equals() to determine whether an element is already present.

37. What is Iterator?

Iterator provides a standard way to traverse a collection and can support removal through its remove() method where supported.

38. What is multithreading?

Multithreading allows multiple execution paths to make progress within a process. It can improve responsiveness and throughput when shared state is handled safely.

39. What is synchronization?

Synchronization controls concurrent access to shared mutable state using mechanisms such as synchronized, locks and concurrent collections.

40. What is volatile?

volatile provides visibility and ordering guarantees for a variable between threads. It does not make compound operations such as count++ atomic.

41. What is garbage collection?

Garbage collection automatically identifies unreachable objects and reclaims their memory.

42. What is a Java memory leak?

A Java memory leak occurs when objects are no longer logically needed but remain reachable, preventing garbage collection. Examples include unbounded caches and forgotten listeners.

43. What is an enum?

An enum defines a fixed set of named constants and can also contain fields, constructors and methods.

44. What is a lambda expression?

A lambda is a concise implementation of a functional interface, such as (a, b) -> a + b.

45. What is a functional interface?

A functional interface has exactly one abstract method and can be used as the target of a lambda or method reference.

46. What is a Stream?

A Stream provides a declarative pipeline for processing data using operations such as filter, map and collect. It does not itself store the data.

47. map() vs flatMap()?

map transforms each input into one output. flatMap transforms each input into a stream-like result and flattens those results into one stream.

48. Intermediate vs terminal Stream operations?

Intermediate operations such as filter and map are generally lazy. Terminal operations such as collect, reduce and forEach trigger pipeline execution.

49. What is Optional?

Optional represents a value that may or may not be present. It can make absence explicit, particularly in method return types.

50. Why should Optional.get() generally be avoided?

Calling get() without checking presence can throw NoSuchElementException. Prefer orElse, orElseGet, map, flatMap or orElseThrow according to the required behaviour.

51. What is method reference?

A method reference is a compact lambda alternative that refers to an existing method, such as String::trim or System.out::println.

52. What is an immutable object?

An immutable object cannot change its observable state after construction. Immutability simplifies reasoning, caching and thread-safe sharing.

53. How do you make a class immutable?

Use private final fields, initialize all state in the constructor, avoid setters, make defensive copies of mutable values and prevent subclasses from breaking invariants.

54. Composition vs inheritance?

Inheritance represents an is-a relationship. Composition represents a has-a relationship and often gives more flexibility by delegating behaviour to collaborators.

55. What is dependency injection?

Dependency injection supplies an object's dependencies from outside rather than having the object construct them. It reduces coupling and improves testability.

56. What is an interface default method?

A default method is an interface method with an implementation. It allows interfaces to evolve without immediately forcing every implementation to define the new method.

57. What is ConcurrentHashMap?

ConcurrentHashMap is a thread-safe map designed for concurrent access with better scalability than synchronizing an entire HashMap in many workloads.

58. What is ExecutorService?

ExecutorService manages task execution using a pool or other executor strategy. It separates task submission from thread lifecycle management.

59. What is a deadlock?

Deadlock occurs when threads wait indefinitely for resources held by one another. Consistent lock ordering and careful lock design can prevent it.

60. What is the difference between a process and a thread?

A process has its own address space and resources. Threads execute inside a process and normally share its memory, requiring careful synchronization.

Set 2: Professional / 10+ Years Experience — 60 Questions

This section focuses on JVM internals, concurrency, performance, collections, production troubleshooting, API design, architecture and senior engineering judgment.

1. Explain JVM architecture.

The JVM includes class loading, runtime data areas such as heap and thread stacks, an execution engine with interpreter/JIT compilation, native interfaces and garbage collection. Details vary by JVM implementation.

2. Explain class loading, linking and initialization.

Class loading obtains class bytes, linking performs verification, preparation and resolution as needed, and initialization executes static initialization. These stages determine when a type becomes ready for use.

3. What is the parent delegation model?

A class loader normally delegates loading to its parent before attempting the class itself. This promotes consistent type identity and helps prevent application code from replacing platform classes.

4. What is JIT compilation?

JIT compilation converts frequently executed bytecode into optimized native code at runtime using profiling information. Optimizations can include inlining and devirtualization.

5. What is escape analysis?

Escape analysis determines whether objects or values escape a method or thread. The JVM can use this information for optimizations such as scalar replacement and synchronization elimination in suitable cases.

6. Explain Java Memory Model.

The Java Memory Model defines visibility, ordering and atomicity rules for interactions between threads. Happens-before relationships provide the formal basis for many of these guarantees.

7. What is happens-before?

Happens-before defines an ordering in which effects of one action are guaranteed visible to another. Examples include monitor unlock/lock, volatile write/read and thread start/join relationships.

8. volatile vs synchronized?

volatile provides visibility and ordering for a variable but not mutual exclusion or general compound-operation atomicity. synchronized provides mutual exclusion and memory visibility around monitor operations.

9. What is CAS?

Compare-And-Set atomically updates a value only if it still equals an expected value. It is a building block for many non-blocking concurrent algorithms.

10. AtomicInteger vs synchronized counter?

AtomicInteger can perform simple atomic updates without a monitor and may scale well for low-contention state. synchronized is more suitable when multiple related variables or invariants must be updated as one critical section.

11. What is deadlock and how do you prevent it?

Deadlock occurs when threads wait cyclically for locks. Prevention techniques include consistent lock ordering, reducing lock scope, avoiding nested locks where possible and using timed lock acquisition when appropriate.

12. What is livelock?

Livelock occurs when threads are active and repeatedly react to one another but fail to make progress. Randomized backoff or clearer ownership rules can help.

13. What is starvation?

Starvation occurs when a thread cannot obtain a needed resource because other threads continually win access. Fairness, bounded waiting and appropriate scheduling/locking strategies can help.

14. ExecutorService vs manual thread creation?

ExecutorService provides pooling, task management, lifecycle control and result handling. Creating one thread per task can lead to excessive resource usage and poor scalability.

15. Explain ThreadPoolExecutor.

ThreadPoolExecutor manages workers, task queues and rejection. Important parameters include core size, maximum size, keep-alive time, queue type and rejection policy. These should reflect workload and downstream capacity.

16. How do you size a thread pool?

Start from workload characteristics: CPU-bound tasks generally need roughly processor-scale concurrency, while I/O-bound workloads can tolerate more. Measure utilization, queueing, latency and downstream limits before tuning.

17. Future vs CompletableFuture?

Future offers asynchronous result retrieval with limited composition. CompletableFuture supports chaining, combination, callbacks and structured exception handling for asynchronous workflows.

18. What is ForkJoinPool?

ForkJoinPool is optimized for tasks that recursively split into subtasks and uses work stealing so idle workers can process tasks from other workers.

19. What are virtual threads?

Virtual threads are lightweight JVM-managed threads suited to high-concurrency blocking I/O. They improve scalability for many waiting tasks but do not make CPU-bound computation faster.

20. Explain HashMap resizing.

When entries exceed a threshold based on capacity and load factor, the table expands and entries are redistributed according to the new capacity. Pre-sizing can reduce resize overhead when expected size is known.

21. Why are mutable HashMap keys dangerous?

If fields used by equals() or hashCode() change after insertion, the map may no longer find the key in the expected bucket. Immutable keys are strongly preferred.

22. HashMap vs ConcurrentHashMap?

HashMap is not safe for concurrent structural updates. ConcurrentHashMap provides thread-safe concurrent operations using fine-grained coordination and non-blocking techniques where appropriate.

23. ArrayList vs CopyOnWriteArrayList?

ArrayList is not thread-safe. CopyOnWriteArrayList creates a new array for each mutation, making reads/iteration efficient when writes are rare but potentially expensive for write-heavy workloads.

24. What is a weakly consistent iterator?

Concurrent collection iterators can tolerate concurrent modifications and do not necessarily reflect every update. They generally avoid fail-fast ConcurrentModificationException behaviour.

25. How do you design an immutable class?

Use private final state, complete constructor initialization, no mutators, defensive copies for mutable inputs/outputs, and prevent subclass-based state changes when necessary.

26. What are records?

Records provide concise data-carrier classes with generated accessors, constructor, equals, hashCode and toString. Their immutability is shallow because referenced objects can still be mutable.

27. What are sealed classes?

Sealed classes and interfaces restrict which types may directly extend or implement them. They are useful for controlled domain hierarchies and exhaustive reasoning.

28. Explain modern pattern matching.

Pattern matching can combine type checking with variable binding and modern switch constructs can make type-based branching more expressive, depending on the Java version.

29. When should parallel streams be avoided?

Avoid them for tiny tasks, blocking I/O, shared mutable state, strict ordering requirements or when common-pool contention could hurt the application. Benchmark before adopting them.

30. What is stream laziness?

Intermediate Stream operations build a pipeline but normally do not execute until a terminal operation consumes the stream. This enables efficient traversal and short-circuiting.

31. Explain Optional design guidelines.

Optional is most useful as a return type representing possible absence. Avoid using it indiscriminately for fields, parameters or serialization models, and avoid unsafe get() calls.

32. What is defensive copying?

Defensive copying prevents callers from changing internal mutable state by copying values when accepting or returning mutable objects.

33. How does garbage collection work at a high level?

GC identifies objects that are no longer reachable and reclaims memory. Different collectors use different strategies to balance throughput, memory footprint and pause-time goals.

34. How would you investigate an OutOfMemoryError?

Identify the exact error and memory region, inspect GC logs and metrics, capture a heap dump where appropriate, analyze retained objects and identify unbounded caches, collections or unexpected references.

35. How would you investigate a memory leak?

Confirm growth over time, compare heap dumps, inspect dominators and retained sizes, trace objects to GC roots and identify retention caused by caches, listeners, ThreadLocals or static state.

36. How would you investigate high CPU?

Identify hot threads, capture thread dumps and profiling data, correlate CPU with application activity, then inspect algorithms, loops, synchronization, allocations and GC.

37. How would you investigate long GC pauses?

Analyze collector logs, pause durations, allocation rates, heap occupancy, promotion and live-set size. Determine whether collector choice, heap sizing or application allocation/retention is responsible.

38. What is a thread dump used for?

Thread dumps show thread states and stack traces. They help diagnose blocked threads, deadlocks, CPU hotspots, pool exhaustion and unexpected waiting.

39. What is ThreadLocal and its risk in pools?

ThreadLocal stores thread-specific values. In pooled threads, values can survive task completion and retain objects or context unexpectedly, so values should be removed when appropriate.

40. What is a memory leak caused by ThreadLocal?

A pooled worker thread may retain a ThreadLocal value after a task finishes. If the value references a large object graph, that graph may remain reachable as long as the worker thread lives.

41. Explain SOLID principles.

SOLID represents Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion. Together they encourage maintainable, loosely coupled designs.

42. Composition vs inheritance in large systems?

Composition usually limits coupling and makes behaviour replaceable. Inheritance is useful when substitutability and shared implementation are genuinely part of the domain model.

43. What design patterns do you commonly use?

Useful patterns include Strategy, Factory, Builder, Adapter, Decorator, Proxy, Observer and Template Method. Senior engineers should choose patterns based on actual design problems, not pattern popularity.

44. How would you implement a thread-safe Singleton?

Common options include an enum singleton or initialization-on-demand holder. Double-checked locking can also work when the instance is volatile and construction is correctly guarded.

45. How do you design a high-throughput Java service?

Define throughput and latency goals, minimize contention and unnecessary allocation, use appropriate concurrency and pooling, control backpressure, optimize I/O and measure CPU, GC and downstream dependencies.

46. How do you improve Java performance?

Profile first. Optimize the actual bottleneck, which may be algorithms, database calls, serialization, allocation, locking or I/O. Validate improvements with representative benchmarks.

47. How do you reduce object allocation?

Reuse objects only where it is safe and beneficial, avoid unnecessary temporary objects, choose appropriate data structures, reduce boxing and excessive intermediate Stream objects, and verify with allocation profiling.

48. How do you identify lock contention?

Use thread dumps and profiling/JFR data to identify blocked or contended monitors and locks. Inspect critical-section duration, lock granularity and whether shared mutable state can be redesigned.

49. What is backpressure?

Backpressure prevents producers from overwhelming consumers by limiting or regulating the rate of work. It is important in queues, streaming systems and high-throughput services.

50. What is graceful shutdown?

Graceful shutdown stops accepting new work, allows in-flight work to complete within a deadline, closes resources and terminates executors cleanly. Forced shutdown is a fallback after the grace period.

51. How do you handle exceptions in a large Java service?

Use meaningful domain exceptions, preserve root causes, avoid swallowing errors, separate recoverable from unrecoverable failures, log with appropriate context and expose stable error contracts at service boundaries.

52. Checked vs unchecked exceptions in enterprise applications?

Use checked exceptions selectively when callers are genuinely expected to recover. Unchecked exceptions are often preferable for programming errors and many application-layer failures where recovery is handled at higher boundaries.

53. What makes a good Java API?

A good API has clear contracts, stable abstractions, sensible naming, controlled mutability, predictable error handling, compatibility considerations and minimal exposure of implementation details.

54. What is API compatibility?

Source compatibility means existing source can compile against the new version. Binary compatibility concerns already-compiled clients. Behavioural compatibility concerns whether existing programs continue to behave as expected.

55. How do you review Java code as a senior engineer?

Review correctness, design, concurrency, error handling, resource lifecycle, security, performance, testing and maintainability. Also question whether the implementation solves the business problem without unnecessary complexity.

56. How do you approach technical debt?

Classify debt by business and technical impact, quantify risks where possible, prioritize high-leverage fixes and integrate debt reduction into normal delivery rather than treating all debt as equally urgent.

57. How do you troubleshoot a production Java incident?

Stabilize the system first, assess impact, inspect metrics/logs/traces, form hypotheses, gather evidence, mitigate safely, then perform root-cause analysis and implement preventive actions.

58. What is observability for Java services?

Observability combines metrics, logs and traces to explain system behaviour. JVM metrics such as heap, GC, threads and CPU should be correlated with application and dependency telemetry.

59. How do you decide between synchronous and asynchronous processing?

Use synchronous processing when immediate response and simple flow are appropriate. Prefer asynchronous processing when work is long-running, bursty or can be decoupled, provided consistency, retries and failure handling are designed carefully.

60. What differentiates a 10-year Java engineer from a junior developer?

Senior engineers demonstrate judgment: they make trade-offs, understand JVM and concurrency behaviour, design scalable systems, diagnose production failures, manage technical debt, review architecture and code, mentor others and connect technical decisions to business outcomes.